AI CLUSTERS

Model-Based Testing for Open-Source Software Components: Public Modeling Library and Interaction-Level Validation

Mengqi Liu *(Alibaba)*

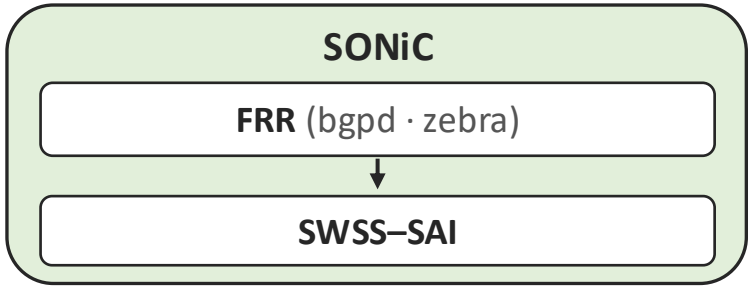
Ali Kheradmand *(Google)*

OCP PLATINUM Member

AI Fabrics Run on Open-Source Code

- Correctness matters: incorrect recovery from a link flap can stall thousands of GPUs.
- Validation gap: upstream test suites may miss deployment-specific behavior.
  - Design intent is spread across contributors, RFCs, and HLDs.
- Existing approaches do not **guarantee** comprehensive validation.
  - Manually-developed tests are valuable, but labor intensive and insufficient.
  - LLMs do not guarantee comprehensive enumeration.

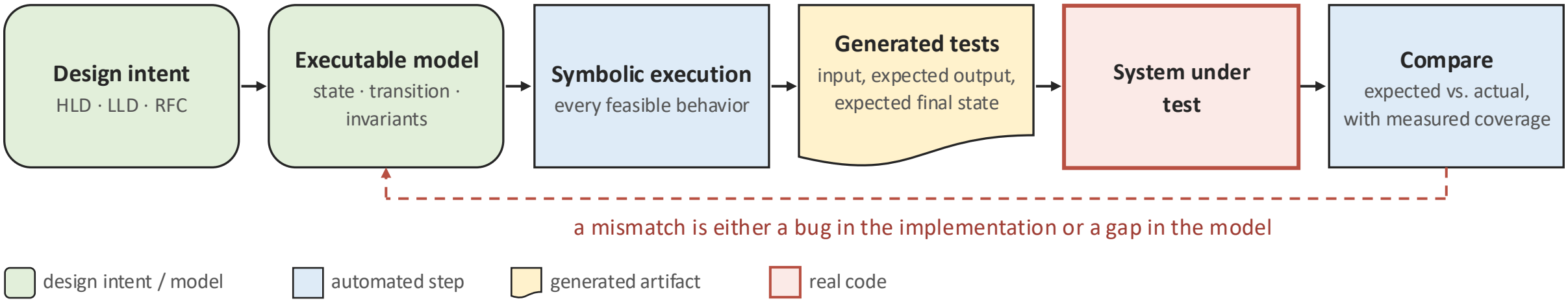
This talk targets **SONiC**; the methodology generalizes



Maintain **models** in addition to hand-written tests. The model keeps up with upstream, and the suites are generated from it with coverage guarantees.

Model-Based Testing

- A model is an executable spec of the **design intent** (HLD/LLD)
  - Abstract / Conceptual state definitions and transitions logic, not a transcription of the implementation.
- Everything downstream is mechanical with **mathematically guaranteed coverage**.



*SRv6 in frr/zebra: 6 API models, ~1,000 lines of model code, **90%+ code coverage**, **10+ real bugs** found in production software.*

Open Repo: sonic-formal-infra

- Apache 2.0, maintained by the SONiC Community: the library, the models, and generated suites.
- Modeling library: typed primitives; solver-friendly by construction.
- Models and examples: real models, end to end.
  - bgpd nexthop tracking — 468 cases; BGP malformed packets; BGP-LS; SAI.
  - JSON suites + gtest drivers.
- Contribution paths: **leverage existing models to build new ones.**
  - Adapt models for components you use; share bugs found by the suites.
- Recent run: **2,000+ automatically generated tests** checked FreeRangeRouting (FRR)'s BGP UPDATE error handling against RFC 7606; **found justified differences.**

Python Modeling Library: Control Plane SW

```python
# Opaque IDs are int aliases, not str: the model only
# compares identity, so the solver never reasons about
# symbolic strings.
opaque_addr_t = NewType('opaque_addr_t', int)

def to_ipv6_opaque_address(a: opaque_addr_t) -> str:
    """Deterministic address for a real harness."""
    h = blake2b(str(a).encode(), digest_size=16)
    ...

# Flat (addr, length) tuple: cheap for CrossHair,
# readable in model code.
class prefix_t(NamedTuple):
    addr:   addr_t
    length: int

# Constant block table per mask length, so a prefix
# match is two integer comparisons.
def prefix_match(p: prefix_t, a: addr_t) -> bool:
    block = _BLOCK[proto][p.length]
    return p.addr <= a < p.addr + block
```

- **Domain types: solver-friendly and human-readable.**
  - Opaque IP addr as integer: identity comparison only; lowered to concrete address for test execution.
  - Fixed-width aliases (`uint8_t`, `uint16_t`, …): mirror C types.
- **Primitive functions (e.g., prefix_match):** reduced solver overhead.
- Takeaway: a statically-typed Python subset: type checkable, executable, and a specification the community can actually review.

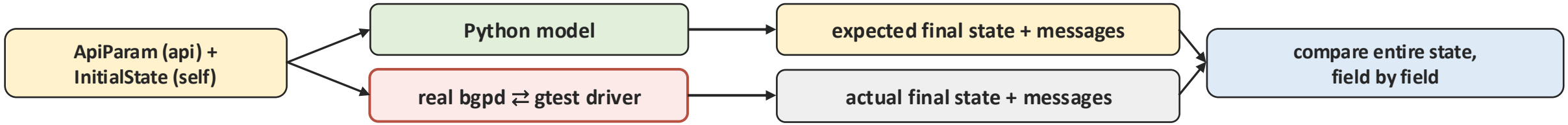
Example: Nexthop Tracking

Modeling the transition — api_bgp_nh_update.py

```
def api_bgp_nh_update(self, api) -> DownstreamMsgs:
    res = DownstreamMsgs()
    if not any(bnc.endpoint == api.endpoint
               for bnc in self.nexthop_cache_table):
        return res                      # [1] skipped
    for bnc in self.nexthop_cache_table:
        if bnc.endpoint == api.endpoint:
            if bnc.resolved == api.resolved \
               and bnc.valid == api.status_up:
                return res              # [2] skipped
            bnc.valid = api.status_up   # [3] executed
            bnc.resolved = api.resolved; break
    for r in self.rib:
        if any(nh == api.endpoint for nh in r.nexthops):
            if self._all_nhs_down(r):
                res.emit_del(r.prefix)      # skipped
            else:
                up = [nh for nh in r.nexthops
                      if self._is_valid_nh(nh)]
                res.emit_add(r.prefix, up) # [4] executed
    return res
```

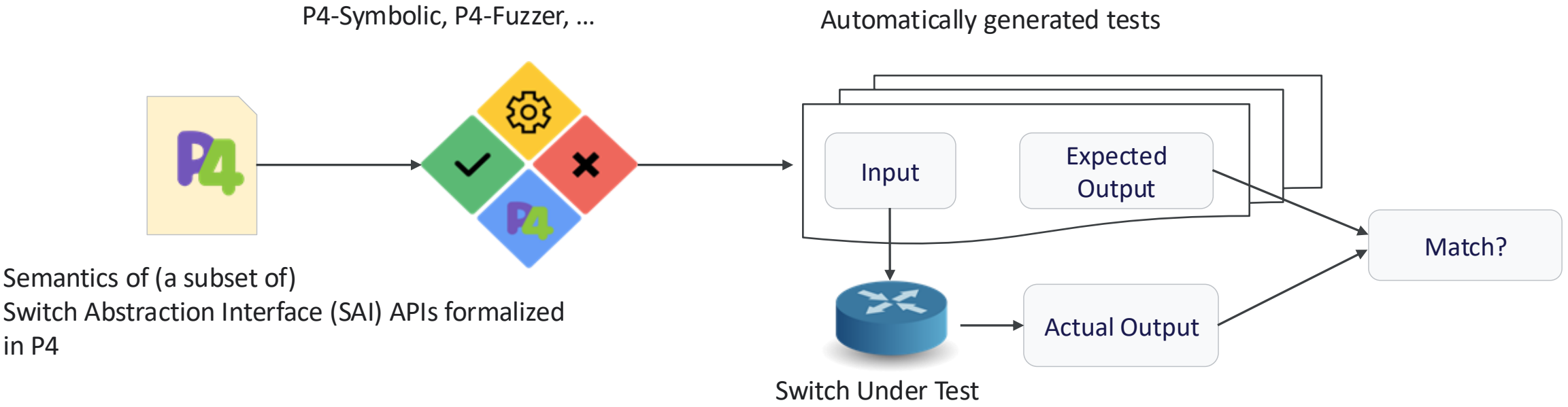
Case #9 of 468 — nh_update_formatted.json

```
{ "TestId": 9,  "Op": "api_bgp_nh_update",
  "InitialState": {
    "nexthop_cache_table": [
      { "endpoint": "…::306f", "resolved": "…::306f",
        "valid": false },
      { "endpoint": "…::8f5f", "resolved": "…::8f5f",
        "valid": false } ],
    "rib": [ { "prefix": "::/0",
               "nexthops": [ "…::306f", "…::8f5f" ] } ] },
  "ApiParam": {
    "endpoint":  "…::8f5f",     ← [1] bnc in cache → skip return
    "resolved":  "…::53e7",     ← [2] resolved differs → skip return
    "status_up": true },        ← [3] update valid + resolved
  "FinalState": {
    "nexthop_cache_table": [
      { …::306f unchanged },
      { "endpoint": "…::8f5f", "resolved": "…::53e7",
        "valid": true } ],      ⇐ [3] valid + resolved checked
    "rib": … unchanged … },
  "DownstreamMsgs": [ { "op": "SET", "prefix": "::/0",
    "nexthops": [ "…::8f5f" ] } ] }  ⇐ [4] SET with valid …::8f5f
```



The model guides the input selection and serves as the test oracle.

Case Study: P4-Based Automated Reasoning



Semantics of (a subset of) Switch Abstraction Interface (SAI) APIs formalized in P4

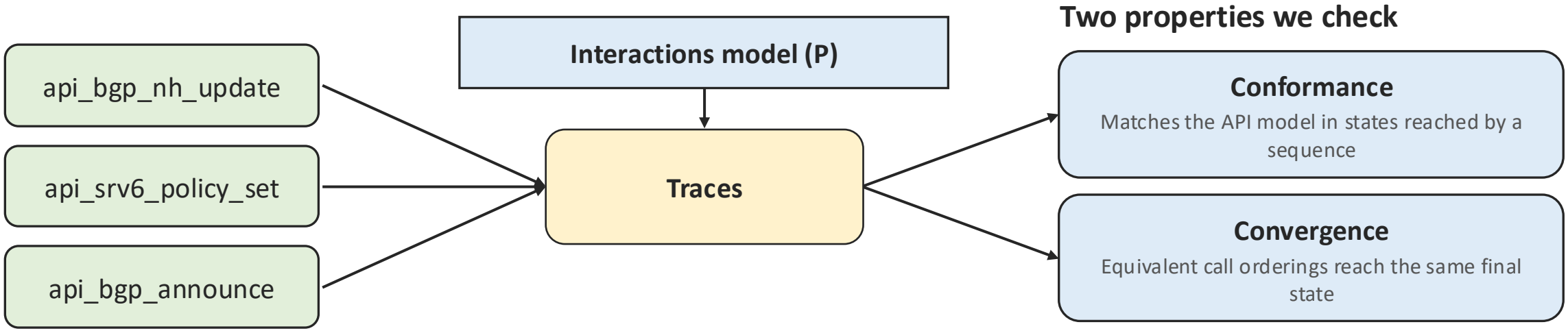
300+ (unique) bugs found so far (and many more bugs prevented)

SwitchV: Automated SDN Switch Validation with P4 Models (SIGCOMM'22)

Kinan Dak Albab, Jonathan Dilorenzo, Stefan Heule, Ali Kheradmand, Steffen Smolka, Konstantin Weitz, Muhammad Tirmazi, Jiaqi Gao, Minlan Yu

Composition: Interaction-Level Validation

- Each API can pass unit tests and still break in **composition** — sequences reach states no unit test starts from, and orderings the APIs were never checked against.
- How to test automatically:
  - an **interactions model** (P Language) enumerates traces, i.e., orderings of API calls.
  - Each step **instantiates a per-API model** — symbolic execution yields concrete tests.



From P Model to Grounded Trace

- P enumerates the orderings; the per-API models turn one pattern into an executable trace.

P model: each module's high-level event handling

```
// Zebra.p — tags and bookkeeping elided
on eZreadSrv6PolicySet do (tag: tMsgTag) {
    if ($) {          // chosen: true
        if ($) {      // chosen: true (one update)
            send bgpd, eNexthopUpdate,
                (tag=…, origin=OPolicySet);
        } else {      // alternative: two updates
            send bgpd, eNexthopUpdate, (…);
            send bgpd, eNexthopUpdate, (…);
        }
    }
}
```

Generated Python trace: grounded in the API models

```
# Abbreviated; imports/preconditions omitted
def trace(self: TraceState, policy: ZApiPolicy):
    res_0 = self.zebra.api_srv6_policy_set(policy)
    p0 = len(res_0) == 1
    assume(p0)    # one nh_update
    nh = to_bapi_nexthop_update(res_0[0])
    res_1 = self.bgpd.api_bgp_nh_update(nh)
    p1 = (len(res_1.del_msgs) == 0
          and len(res_1.add_msgs) == 1)
    assume(p1)              # one route_add
    route = to_zapi_route_add(res_1.add_msgs[0])
    self.zebra.api_route_add(route)
```

Selected sequence
ZEBRA_POLICY_SET → BGPD_NH_UPDATE → ZEBRA_ROUTE_ADD

→

Trace function
one per distinct trace, with post-condition checks

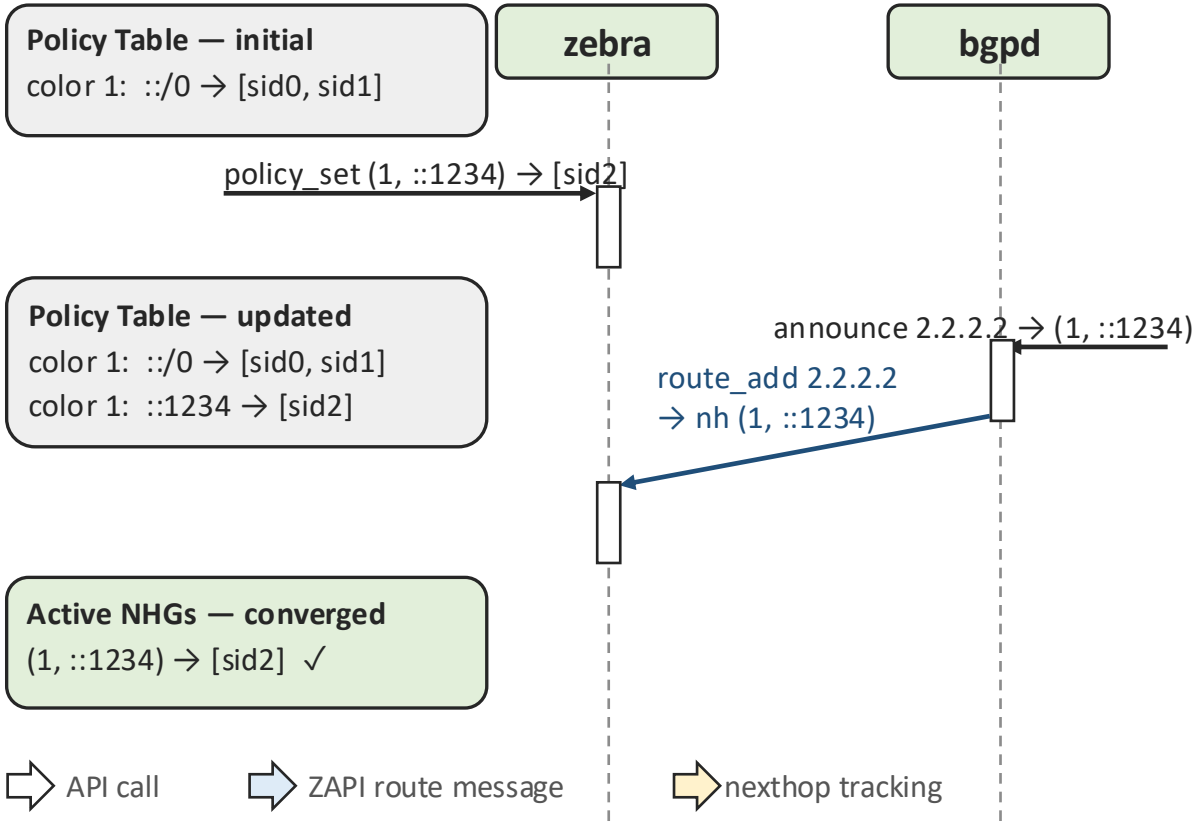
→

Symbolic search
concrete inputs that realize this trace

→

Test case
inputs + expected messages, replayed on real FRR

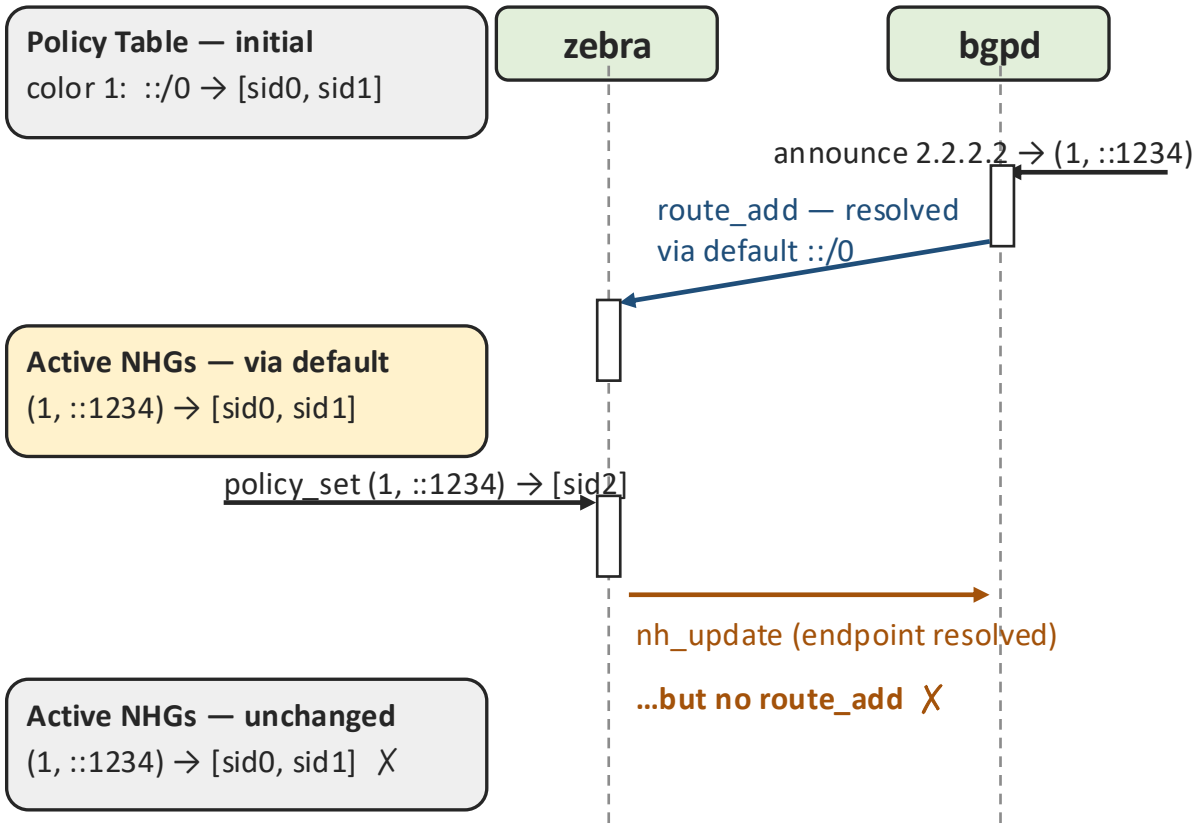
Interaction Bug 1/2: Typical Order



Policy first, then route announce

1. `policy_set` installs `[sid2]` for (color 1, endpoint `::1234`).
2. bgpd announces `2.2.2.2` with nexthop `(1, ::1234)`; zebra resolves it against the updated policy.
3. Active NHGs = **[sid2]** ✓ — forwarding matches intent.

Interaction Bug 2/2: Reversed Order



Reversed order, same two calls

1. bgpd announces first; zebra resolves the nexthop via the default policy `::/0 → [sid0, sid1]`.
2. `policy_set` then installs `[sid2]`; zebra notifies bgpd with `nh_update`.
3. No `route_add` follows, so the active nexthop groups are never refreshed.
4. Active NHGs stay **[sid0, sid1]** ✗ — forwarding is silently stale.

A **convergence** bug: same calls, different final states.

The same intended configuration leaves different active NHGs: [sid2] versus [sid0, sid1].

LLM Complements Formal Methods

1. Model faithfulness

- LLM reviews handwritten models.
  - **Correctness**: does the model say what the design documents intend?
  - **Completeness**: is every behavior covered, e.g. each RFC error path?
- Adoption: AI-assisted checking lowers the cost of both writing and reviewing a model.

2. Exploration acceleration

- The interactions model is deliberately concise, hence over-approximate: it enumerates traces that turn out to be infeasible once grounded against the per-API models.
- LLMs prove such traces infeasible cheaply, providing deductive reasoning arguments.

The model and the generated tests remain the arbiter of correctness. AI accelerates the search and adoption; it does not replace the check.

Call to Action

- Key open-source components need validation with model-coverage guarantees: add **formal models** to reach cases that existing tests miss.
- **Adopt: github.com/sonic-net/sonic-formal-infra** – modeling library, reusable models, test harness, and generated suites, Apache-2.0.
- sonicfoundation.dev · opencompute.org/projects/networking

Thank You!

